



Transformers

Os Transformers são uma arquitetura de Redes Neurais baseada no mecanismo **Attention**.

Esta abordagem foi proposta em 2017 no artigo "[Attention Is All You Need](#)" e desde então revolucionou o campo da Inteligência Artificial.

Uma das maiores provas do impacto dos Transformers é o surgimento de modelos como:

- **GPT (Generative Pre-trained Transformer)**
- **BERT (Bidirectional Encoder Representations from Transformers)**

O objetivo deste Google Colab é explorar de forma prática a arquitetura dos Transformers.

Ao longo do notebook, iremos percorrer as suas principais componentes construindo gradualmente a intuição por trás do modelo.

```
In [ ]: !wget https://raw.githubusercontent.com/Eliezer-Carvalho/Tiny-GPT/refs/heads/main/data/Dataset%20Tiny%20Stories.txt
--2026-05-12 19:42:18-- https://raw.githubusercontent.com/Eliezer-Carvalho/Tiny-GPT/refs/heads/main/data/Dataset%20Tiny%20Stories.txt
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 12165003 (12M) [text/plain]
Saving to: 'Dataset Tiny Stories.txt'

Dataset Tiny Storie 100%[=====>] 11.60M --.-KB/s in 0.1s

2026-05-12 19:42:18 (109 MB/s) - 'Dataset Tiny Stories.txt' saved [12165003/12165003]
```

1. Tokenização

Os Transformers consistem em operações sobre números.

Para tal, deve haver primeiro um mapeamento de qualquer input para representação numérica.

A **Tokenização** divide o texto em pequenos fragmentos chamados **tokens**.

Um token pode ser uma palavra inteira, parte de uma palavra ou até mesmo um sinal de pontuação.

A forma como essa divisão é feita depende principalmente dos **Algoritmos de Tokenização**.

Entre os mais comuns estão:

- **BPE (Byte Pair Encoding)**
- **Unigram**
- **SentencePiece**
- **WordPiece**
- **Word-Level Tokenization**
- **Character-Level Tokenization**

Diferentes algoritmos produzem diferentes tokenizações, o que pode impactar o desempenho do modelo.

Existem também os **Pré Tokenizers** que aplicam uma transformação especial no dataset antes de aplicar o Algoritmo de Tokenização.

Pela minha experiência, nem sempre um Pré Tokenizer traz vantagens.

Análise do Dataset

```
In [ ]: import re      #https://pypi.org/project/regex/

with open ("Dataset Tiny Stories.txt", "r", encoding = "utf-8") as f:
    dataset = f.read()

#https://regexbox.com/cheatsheet
print (f"Caractères --> {len(dataset)}")
print (f"Palavras --> {len(re.findall(r'\w+', dataset))}")
print (f"Linhas --> {len(re.findall(r'\n', dataset))}")
```

Caractères --> 12165003

Palavras --> 2447271

Linhas --> 81035

Tokenizer

Neste projeto, criamos o nosso próprio Tokenizer de raiz!

Não é algo obrigatório, é possível usar Tokenizers já existentes.

Esta ideia surgiu, porque para cada dataset poderá ser necessário adaptar um Tokenizer para conseguir ter uma boa Tokenização.

É fulcral entender que esta etapa, apesar de por vezes ser passada ao lado, é uma **etapa de extrema importância, pois define a estrutura da Linguagem que o modelo vai aprender.**

No meu caso em específico, era perfeitamente possível utilizar um Tokenizer comum (*gpt* ou *bert*) pois estes são treinados com gigas e mais gigas de datasets predominantemente da Língua Inglesa porém **Where's the fun ? ;)**

Se tivesse um dataset mais específico com Linguagens antigas ou até mesmo Línguas como o Chinês, **seria obrigatório criar um Tokenizer de raiz.**

E a ideia que quero deixar, é esta:

A Tokenização é uma fase bastante importante em modelos de Processamento de Linguagem Natural. Consoante o dataset e o tipo de objetivo, é necessário fazer uma análise e avaliação de qual é a melhor opção a tomar.

```
In [ ]: from tokenizers import Tokenizer #https://huggingface.co/docs/tokenizers/index
from tokenizers.models import BPE #https://huggingface.co/docs/tokenizers/api/
from tokenizers.trainers import BpeTrainer #https://huggingface.co/docs/tokenizers/trainers
from tokenizers.pre_tokenizers import Metaspace, Punctuation, Whitespace, Whitespace
import matplotlib.pyplot as plt
```

Tokenização não é algo Perfeito!

Como é que eu sei que o meu Tokenizer funciona ? Que é o adequado para o meu dataset? Como posso avaliar o meu Tokenizer ?

Não há uma resposta correta! Mas há maneiras de chegar a um bom resultado!

O Algoritmo de Tokenização que selecionei para este estudo foi o **BPE** , tentei também aplicar **Unigram** mas por motivos de custos computacionais não foi possível prosseguir.

O estudo consiste na análise de vários Tokenizers BPE com diferentes técnicas de Pré-Tokenizers e números de Vocab_Size distintos.

A maneira como avaliei estes Algoritmos baseia-se em duas métricas:

- **Fertility**

Fertility é definida como o número médio de tokens necessários para representar uma palavra.

Tem de ser SEMPRE > 1 , se for < 1 , o Tokenizer não serve!

Pode ser representada matematicamente como:

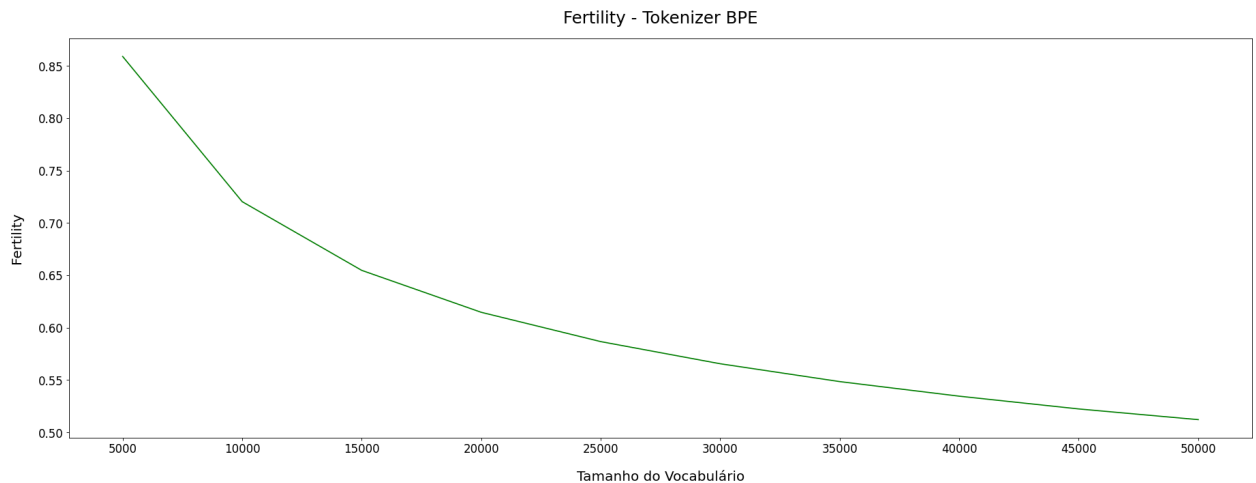
$$\frac{\text{Número de Tokens}}{\text{Número de Palavras no Dataset}}$$

- **Coverage Visual**

Baseia-se na capacidade de observar se o Tokenizer consegue transformar uma frase de teste em tokens de maneira eficiente!

BPE

In []: t



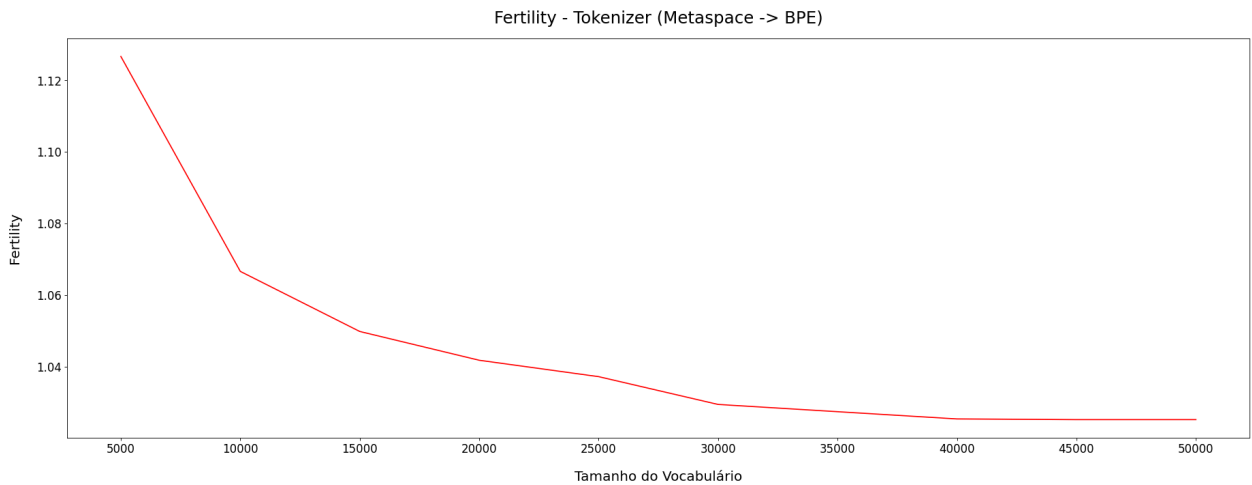
TOKENIZAÇÃO DOS DIFERENTES TOKENIZERS BPE

One day, a boy named Tim went to the park to play. He saw his friend, Sam, playing with a toy car.

(BPE, 5000) -> ['One day, a boy named Tim ', 'went to the park to play', '. He', 'saw his ', 'friend, Sam, ', 'playing ', 'with a ', 'toy car', '.']
(BPE, 10000) -> ['One day, a boy named Tim ', 'went to the park to play', '. He', 'saw his ', 'friend, Sam, ', 'playing with a ', 'toy car', '.']
(BPE, 15000) -> ['One day, a boy named Tim ', 'went to the park to play', '. He saw his ', 'friend, Sam, ', 'playing with a ', 'toy car', '.']
(BPE, 20000) -> ['One day, a boy named Tim ', 'went to the park to play', '. He saw his ', 'friend, Sam, ', 'playing with a ', 'toy car', '.']
(BPE, 25000) -> ['One day, a boy named Tim ', 'went to the park to play', '. He saw his ', 'friend, Sam, ', 'playing with a ', 'toy car', '.']
(BPE, 30000) -> ['One day, a boy named Tim ', 'went to the park to play', '. He saw his ', 'friend, Sam, ', 'playing with a ', 'toy car', '.']
(BPE, 35000) -> ['One day, a boy named Tim ', 'went to the park to play', '. He saw his ', 'friend, Sam, ', 'playing with a ', 'toy car', '.']
(BPE, 40000) -> ['One day, a boy named Tim ', 'went to the park to play', '. He saw his ', 'friend, Sam, ', 'playing with a ', 'toy car', '.']
(BPE, 45000) -> ['One day, a boy named Tim ', 'went to the park to play', '. He saw his ', 'friend, Sam, ', 'playing with a toy car', '.']
(BPE, 50000) -> ['One day, a boy named Tim ', 'went to the park to play', '. He saw his ', 'friend, Sam, ', 'playing with a toy car', '.']

Metaspace -> BPE

In []: t



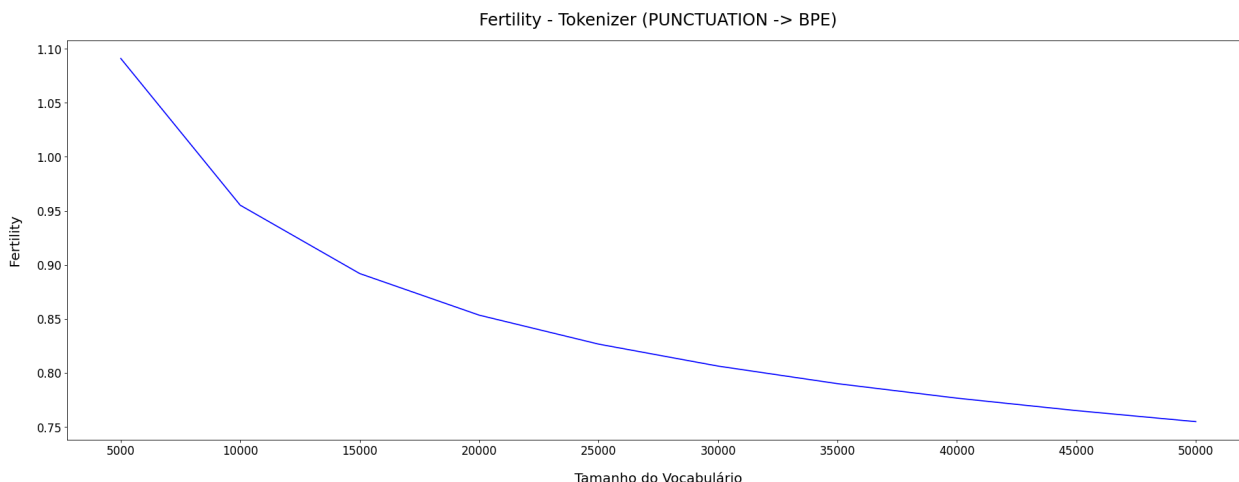
TOKENIZAÇÃO DOS DIFERENTES TOKENIZERS METASPACE -> BPE

One day, a boy named Tim went to the park to play. He saw his friend, Sam, playing with a toy car.

```
(Metaspace -> BPE, 5000) -> ['_One', '_day,', '_a', '_boy', '_named', '_Tim',
'_went', '_to', '_the', '_park', '_to', '_play.', '_He', '_saw', '_his', '_frie
nd,', '_Sam,', '_playing', '_with', '_a', '_toy', '_car.']]
(Metaspace -> BPE, 10000) -> ['_One', '_day,', '_a', '_boy', '_named', '_Tim',
'_went', '_to', '_the', '_park', '_to', '_play.', '_He', '_saw', '_his', '_frie
nd,', '_Sam,', '_playing', '_with', '_a', '_toy', '_car.']]
(Metaspace -> BPE, 15000) -> ['_One', '_day,', '_a', '_boy', '_named', '_Tim',
'_went', '_to', '_the', '_park', '_to', '_play.', '_He', '_saw', '_his', '_frie
nd,', '_Sam,', '_playing', '_with', '_a', '_toy', '_car.']]
(Metaspace -> BPE, 20000) -> ['_One', '_day,', '_a', '_boy', '_named', '_Tim',
'_went', '_to', '_the', '_park', '_to', '_play.', '_He', '_saw', '_his', '_frie
nd,', '_Sam,', '_playing', '_with', '_a', '_toy', '_car.']]
(Metaspace -> BPE, 25000) -> ['_One', '_day,', '_a', '_boy', '_named', '_Tim',
'_went', '_to', '_the', '_park', '_to', '_play.', '_He', '_saw', '_his', '_frie
nd,', '_Sam,', '_playing', '_with', '_a', '_toy', '_car.']]
(Metaspace -> BPE, 30000) -> ['_One', '_day,', '_a', '_boy', '_named', '_Tim',
'_went', '_to', '_the', '_park', '_to', '_play.', '_He', '_saw', '_his', '_frie
nd,', '_Sam,', '_playing', '_with', '_a', '_toy', '_car.']]
(Metaspace -> BPE, 35000) -> ['_One', '_day,', '_a', '_boy', '_named', '_Tim',
'_went', '_to', '_the', '_park', '_to', '_play.', '_He', '_saw', '_his', '_frie
nd,', '_Sam,', '_playing', '_with', '_a', '_toy', '_car.']]
(Metaspace -> BPE, 40000) -> ['_One', '_day,', '_a', '_boy', '_named', '_Tim',
'_went', '_to', '_the', '_park', '_to', '_play.', '_He', '_saw', '_his', '_frie
nd,', '_Sam,', '_playing', '_with', '_a', '_toy', '_car.']]
(Metaspace -> BPE, 45000) -> ['_One', '_day,', '_a', '_boy', '_named', '_Tim',
'_went', '_to', '_the', '_park', '_to', '_play.', '_He', '_saw', '_his', '_frie
nd,', '_Sam,', '_playing', '_with', '_a', '_toy', '_car.']]
(Metaspace -> BPE, 50000) -> ['_One', '_day,', '_a', '_boy', '_named', '_Tim',
'_went', '_to', '_the', '_park', '_to', '_play.', '_He', '_saw', '_his', '_frie
nd,', '_Sam,', '_playing', '_with', '_a', '_toy', '_car.']]
```

Punctuation -> BPE

In []: t



TOKENIZAÇÃO DOS DIFERENTES TOKENIZERS PUNCTUATION -> BPE

One day, a boy named Tim went to the park to play. He saw his friend, Sam, playing with a toy car.

(Punctuation -> BPE, 5000) -> ['One day', ',', ' a boy named Tim', ' went to the park', ' to play', '.', ' He saw', ' his friend', ',', ' Sam', ',', ' playing', ' with a ', 'toy car', '.']

(Punctuation -> BPE, 10000) -> ['One day', ',', ' a boy named Tim', ' went to the park to play', '.', ' He saw his friend', ',', ' Sam', ',', ' playing', ' with a ', 'toy car', '.']

(Punctuation -> BPE, 15000) -> ['One day', ',', ' a boy named Tim', ' went to the park to play', '.', ' He saw his friend', ',', ' Sam', ',', ' playing', ' with a ', 'toy car', '.']

(Punctuation -> BPE, 20000) -> ['One day', ',', ' a boy named Tim', ' went to the park to play', '.', ' He saw his friend', ',', ' Sam', ',', ' playing', ' with a ', 'toy car', '.']

(Punctuation -> BPE, 25000) -> ['One day', ',', ' a boy named Tim', ' went to the park to play', '.', ' He saw his friend', ',', ' Sam', ',', ' playing', ' with a ', 'toy car', '.']

(Punctuation -> BPE, 30000) -> ['One day', ',', ' a boy named Tim', ' went to the park to play', '.', ' He saw his friend', ',', ' Sam', ',', ' playing', ' with a ', 'toy car', '.']

(Punctuation -> BPE, 35000) -> ['One day', ',', ' a boy named Tim', ' went to the park to play', '.', ' He saw his friend', ',', ' Sam', ',', ' playing', ' with a ', 'toy car', '.']

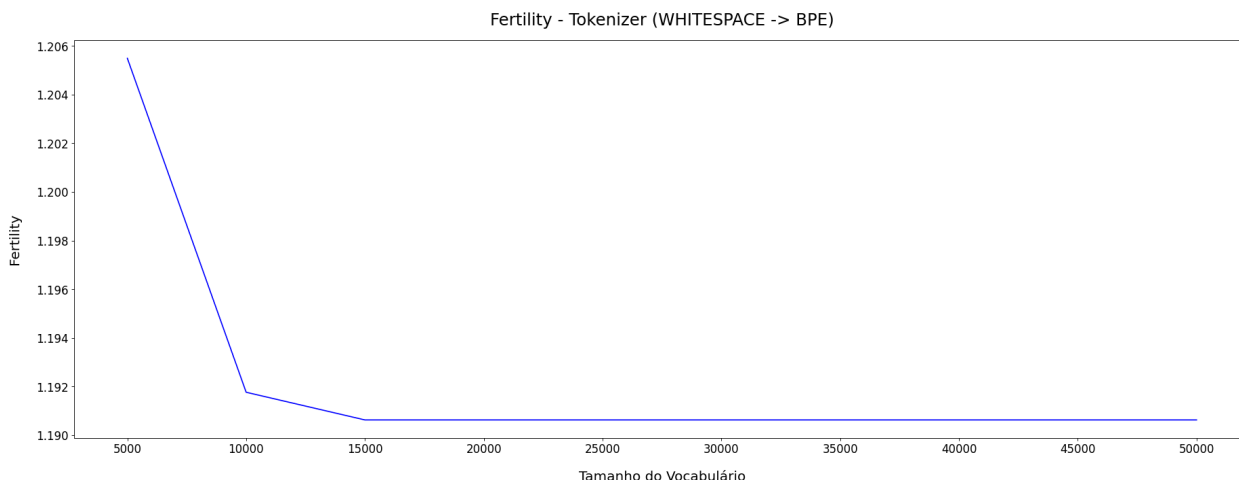
(Punctuation -> BPE, 40000) -> ['One day', ',', ' a boy named Tim', ' went to the park to play', '.', ' He saw his friend', ',', ' Sam', ',', ' playing with a ', 'toy car', '.']

(Punctuation -> BPE, 45000) -> ['One day', ',', ' a boy named Tim', ' went to the park to play', '.', ' He saw his friend', ',', ' Sam', ',', ' playing with a ', 'toy car', '.']

(Punctuation -> BPE, 50000) -> ['One day', ',', ' a boy named Tim', ' went to the park to play', '.', ' He saw his friend', ',', ' Sam', ',', ' playing with a ', 'toy car', '.']

Whitespace -> BPE

In []: t



TOKENIZAÇÃO DOS DIFERENTES TOKENIZERS WHITESPACE -> BPE

One day, a boy named Tim went to the park to play. He saw his friend, Sam, playing with a toy car.

(Whitespace -> BPE, 5000) -> ['One', 'day', ',', ',', 'a', 'boy', 'named', 'Tim', 'went', 'to', 'the', 'park', 'to', 'play', '.', 'He', 'saw', 'his', 'friend', ',', ',', 'Sam', ',', ',', 'playing', 'with', 'a', 'toy', 'car', '.']

(Whitespace -> BPE, 10000) -> ['One', 'day', ',', ',', 'a', 'boy', 'named', 'Tim', 'went', 'to', 'the', 'park', 'to', 'play', '.', 'He', 'saw', 'his', 'friend', ',', ',', 'Sam', ',', ',', 'playing', 'with', 'a', 'toy', 'car', '.']

(Whitespace -> BPE, 15000) -> ['One', 'day', ',', ',', 'a', 'boy', 'named', 'Tim', 'went', 'to', 'the', 'park', 'to', 'play', '.', 'He', 'saw', 'his', 'friend', ',', ',', 'Sam', ',', ',', 'playing', 'with', 'a', 'toy', 'car', '.']

(Whitespace -> BPE, 20000) -> ['One', 'day', ',', ',', 'a', 'boy', 'named', 'Tim', 'went', 'to', 'the', 'park', 'to', 'play', '.', 'He', 'saw', 'his', 'friend', ',', ',', 'Sam', ',', ',', 'playing', 'with', 'a', 'toy', 'car', '.']

(Whitespace -> BPE, 25000) -> ['One', 'day', ',', ',', 'a', 'boy', 'named', 'Tim', 'went', 'to', 'the', 'park', 'to', 'play', '.', 'He', 'saw', 'his', 'friend', ',', ',', 'Sam', ',', ',', 'playing', 'with', 'a', 'toy', 'car', '.']

(Whitespace -> BPE, 30000) -> ['One', 'day', ',', ',', 'a', 'boy', 'named', 'Tim', 'went', 'to', 'the', 'park', 'to', 'play', '.', 'He', 'saw', 'his', 'friend', ',', ',', 'Sam', ',', ',', 'playing', 'with', 'a', 'toy', 'car', '.']

(Whitespace -> BPE, 35000) -> ['One', 'day', ',', ',', 'a', 'boy', 'named', 'Tim', 'went', 'to', 'the', 'park', 'to', 'play', '.', 'He', 'saw', 'his', 'friend', ',', ',', 'Sam', ',', ',', 'playing', 'with', 'a', 'toy', 'car', '.']

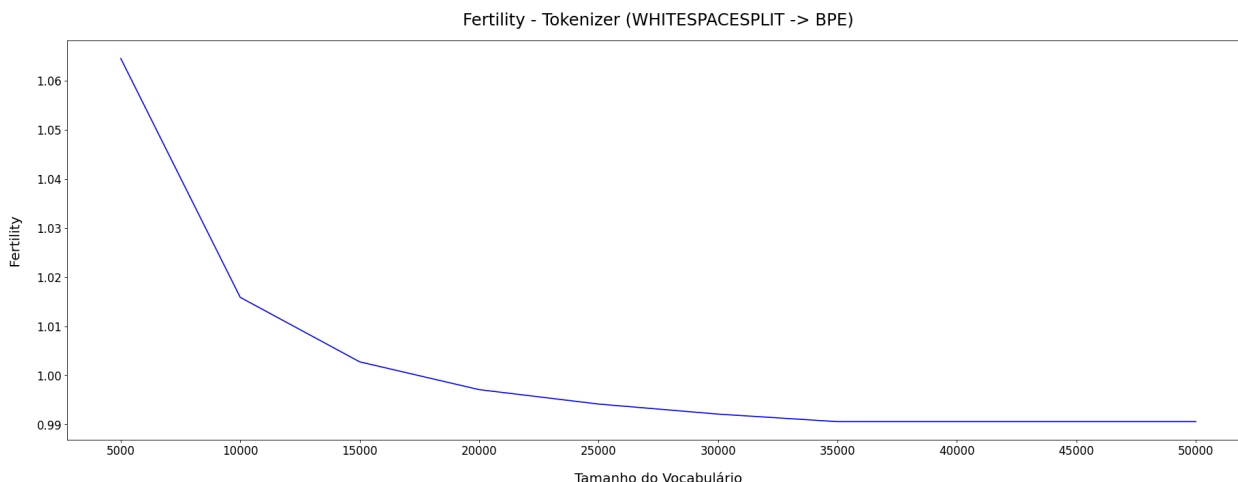
(Whitespace -> BPE, 40000) -> ['One', 'day', ',', ',', 'a', 'boy', 'named', 'Tim', 'went', 'to', 'the', 'park', 'to', 'play', '.', 'He', 'saw', 'his', 'friend', ',', ',', 'Sam', ',', ',', 'playing', 'with', 'a', 'toy', 'car', '.']

(Whitespace -> BPE, 45000) -> ['One', 'day', ',', ',', 'a', 'boy', 'named', 'Tim', 'went', 'to', 'the', 'park', 'to', 'play', '.', 'He', 'saw', 'his', 'friend', ',', ',', 'Sam', ',', ',', 'playing', 'with', 'a', 'toy', 'car', '.']

(Whitespace -> BPE, 50000) -> ['One', 'day', ',', ',', 'a', 'boy', 'named', 'Tim', 'went', 'to', 'the', 'park', 'to', 'play', '.', 'He', 'saw', 'his', 'friend', ',', ',', 'Sam', ',', ',', 'playing', 'with', 'a', 'toy', 'car', '.']

WhitespaceSplit -> BPE

In []: t



TOKENIZAÇÃO DOS DIFERENTES TOKENIZERS WHITESPACESPLIT -> BPE

One day, a boy named Tim went to the park to play. He saw his friend, Sam, playing with a toy car.

(WhitespaceSplit -> BPE, 5000) -> ['One', 'day,', 'a', 'boy', 'named', 'Tim', 'went', 'to', 'the', 'park', 'to', 'play.', 'He', 'saw', 'his', 'friend,', 'Sa m,', 'playing', 'with', 'a', 'toy', 'car.']

(WhitespaceSplit -> BPE, 10000) -> ['One', 'day,', 'a', 'boy', 'named', 'Tim', 'went', 'to', 'the', 'park', 'to', 'play.', 'He', 'saw', 'his', 'friend,', 'Sa m,', 'playing', 'with', 'a', 'toy', 'car.']

(WhitespaceSplit -> BPE, 15000) -> ['One', 'day,', 'a', 'boy', 'named', 'Tim', 'went', 'to', 'the', 'park', 'to', 'play.', 'He', 'saw', 'his', 'friend,', 'Sa m,', 'playing', 'with', 'a', 'toy', 'car.']

(WhitespaceSplit -> BPE, 20000) -> ['One', 'day,', 'a', 'boy', 'named', 'Tim', 'went', 'to', 'the', 'park', 'to', 'play.', 'He', 'saw', 'his', 'friend,', 'Sa m,', 'playing', 'with', 'a', 'toy', 'car.']

(WhitespaceSplit -> BPE, 25000) -> ['One', 'day,', 'a', 'boy', 'named', 'Tim', 'went', 'to', 'the', 'park', 'to', 'play.', 'He', 'saw', 'his', 'friend,', 'Sa m,', 'playing', 'with', 'a', 'toy', 'car.']

(WhitespaceSplit -> BPE, 30000) -> ['One', 'day,', 'a', 'boy', 'named', 'Tim', 'went', 'to', 'the', 'park', 'to', 'play.', 'He', 'saw', 'his', 'friend,', 'Sa m,', 'playing', 'with', 'a', 'toy', 'car.']

(WhitespaceSplit -> BPE, 35000) -> ['One', 'day,', 'a', 'boy', 'named', 'Tim', 'went', 'to', 'the', 'park', 'to', 'play.', 'He', 'saw', 'his', 'friend,', 'Sa m,', 'playing', 'with', 'a', 'toy', 'car.']

(WhitespaceSplit -> BPE, 40000) -> ['One', 'day,', 'a', 'boy', 'named', 'Tim', 'went', 'to', 'the', 'park', 'to', 'play.', 'He', 'saw', 'his', 'friend,', 'Sa m,', 'playing', 'with', 'a', 'toy', 'car.']

(WhitespaceSplit -> BPE, 45000) -> ['One', 'day,', 'a', 'boy', 'named', 'Tim', 'went', 'to', 'the', 'park', 'to', 'play.', 'He', 'saw', 'his', 'friend,', 'Sa m,', 'playing', 'with', 'a', 'toy', 'car.']

(WhitespaceSplit -> BPE, 50000) -> ['One', 'day,', 'a', 'boy', 'named', 'Tim', 'went', 'to', 'the', 'park', 'to', 'play.', 'He', 'saw', 'his', 'friend,', 'Sa m,', 'playing', 'with', 'a', 'toy', 'car.']

Tokenizer Vencedor

```
In [ ]: modelo = Tokenizer(BPE())
```



```

trainer = BpeTrainer (vocab_size = 8000) #Vocab_Size, define o número de tokens

modelo.pre_tokenizer = Metaspace() #Pre-Tokenizer
modelo.train (["Dataset Tiny Stories.txt"], trainer) #Treinar o Algoritmo de Tokenização
modelo.save("Tiny-GPT Tokenizer") #Guardar

tokenizer = Tokenizer.from_file ("Tiny-GPT Tokenizer") #Usar

dataset = tokenizer.encode(dataset) #Encode do dataset

```

Visualização do Melhor Tokenizer

- **Tokens** -> unidades fundamentais de texto que representam *palavras*, *partes de palavras* ou *caracteres*.
- **Ids** -> representação numérica dos tokens

```

In [ ]: print (f"Estatísticas do Tokenizer --> {dataset}")
        print (f"50 Primeiros Tokens --> {dataset.tokens[0:50]}")
        print (f"50 Primeiros Ids --> {dataset.ids[0:50]}")

```

```

Estatísticas do Tokenizer --> Encoding(num_tokens=2643056, attributes=[ids, type_ids, tokens, offsets, attention_mask, special_tokens_mask, overflowing])
50 Primeiros Tokens --> ['_Once', '_upon', '_a', '_time,', '_there', '_was', '_a', '_reliable', '_otter', '_named', '_Ollie.', '_He', '_lived', '_in', '_a', '_river', '_with', '_his', '_family.', '_They', '_all', '_loved', '_to', '_play', '_and', '_swim', '_together.\n', '0', 'ne', '_day,', '_Ollie', "'s", '_mom', '_said,', '_"0', 'll', 'ie,', '_hurry', '_and', '_get', '_some', '_fish', '_for', '_dinner', '!', '_Ollie', '_swam', '_fast', '_to', '_catch']
50 Primeiros Ids --> [256, 261, 80, 263, 230, 105, 80, 3548, 4070, 233, 7355, 136, 468, 138, 80, 1671, 151, 166, 1603, 129, 258, 335, 87, 146, 88, 1290, 803, 38, 1877, 203, 2716, 206, 217, 196, 673, 115, 3477, 5331, 88, 480, 438, 655, 214, 2982, 282, 2716, 1443, 572, 87, 983]

```

Codificador e Decodificador de um Tokenizer

```

In [ ]: teste = tokenizer.encode ("Just do it!")

        print ("Codificador:")
        print (teste.tokens)
        print (teste.ids, "\n")

        print ("Decodificador:")
        print (tokenizer.decode(teste.ids))

```

```

Codificador:
['_Just', '_do', '_it!']
[1370, 201, 2474]

```

```

Decodificador:
_Just _do _it!

```

Espaço Dimensional de um Tokenizer

```
In [ ]: import matplotlib.pyplot as plt

x = dataset.tokens[200:225]
y = dataset.ids[200:225]

plt.figure (figsize = (30, 10), dpi = 80)
plt.scatter (x, y, marker = "x", color = 'blue', s = 500)

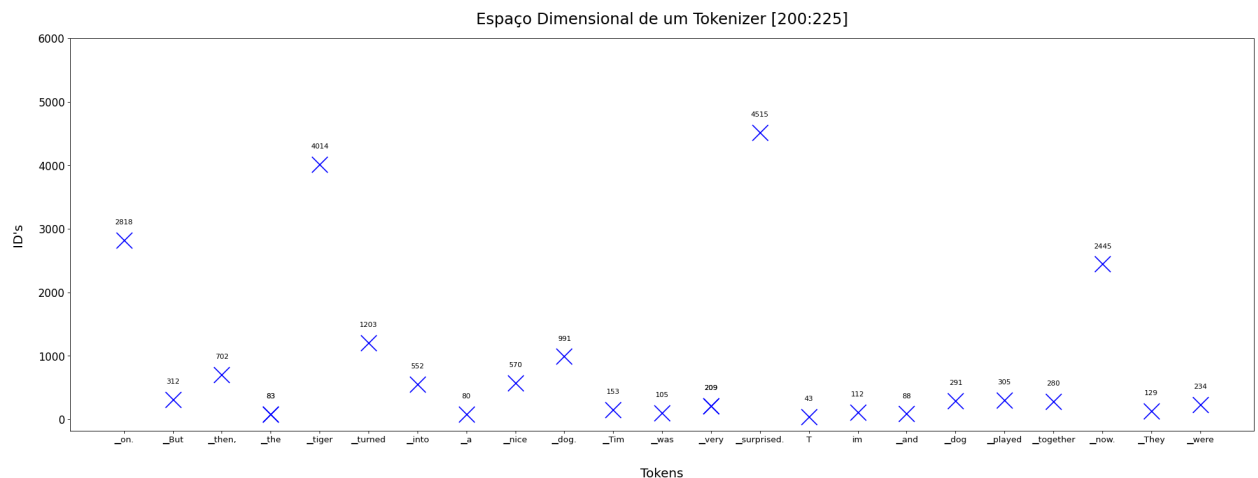
for i in range (len(y)):

    plt.text(x[i], y[i] + 250, y[i], ha = "center", size = 10)

plt.xticks (size = 12)
plt.xlabel ("Tokens", size = 18, labelpad = 20)

plt.yticks (list(range(0, 7000, 1000)), size = 15)
plt.ylabel ("ID's", size = 18, labelpad = 20)

plt.title ("Espaço Dimensional de um Tokenizer [200:225]", size = 22, pad = 20)
plt.show()
```



2. Tensor Shape

Após a Tokenização, o dataset já se encontra convertido para uma representação numérica.

A organização dos dados de entrada é feita através da definição de dois parâmetros fundamentais:

1. **Batch Size** - número de sequências processadas em paralelo durante o treino.

2. **Sequence Length** - número de tokens considerados em cada sequência.

Esta abordagem é necessária porque os Transformers não processam todo o texto de uma vez, mas sim em blocos de tamanho fixo.

De forma análoga à leitura humana, não é possível analisar um texto completo simultaneamente, é necessário dividi-lo em partes para conseguir identificar padrões e relações.

Assim, os dados são representados sob a forma de um tensor com dimensão (B, T).

```
In [ ]: import torch #https://docs.pytorch.org/docs/stable/index.html
import torch.nn as nn #https://docs.pytorch.org/docs/stable/nn.html
import torch.nn.functional as F #https://docs.pytorch.org/docs/stable/nn.funct
```

Definição do Tensor e Dados de Treino vs Dados de Teste

```
In [ ]: tensor = torch.tensor (dataset.ids, dtype = torch.long)
print (len(tensor))
print (tensor [:100])
```

```
#Divisão de Dados de Treino e Dados de Teste
n = int (0.9 * len(tensor))
dados_treino = tensor [:n] #90% dos dados
dados_teste = tensor [n:] #10% dos dados
```

2643056

```
tensor([ 256,  261,   80,  263,  230,  105,   80, 3548, 4070,  233, 7355,  136,
         468,  138,   80, 1671,  151,  166, 1603,  129,  258,  335,   87,  146,
          88, 1290,  803,   38, 1877,  203, 2716,  206,  217,  196,  673,  115,
        3477, 5331,   88,  480,  438,  655,  214, 2982,  282, 2716, 1443,  572,
          87,  983, 1825,  136,  211,  166,  590,   83, 3894, 1243, 2716,  282,
         148,   83, 3894, 1243, 1069,  282,  148, 7355,  328,  619,   87, 5331,
          88,  983,  655,  214,  460, 1603,  225,   46,   58,  436, 2716,  105,
        6716, 3914,  107,  339,   80,  175,  859, 4412,  136, 1676, 1549,  251,
         192,   80, 3914,  238])
```

Tensor (B, T)

```
In [ ]: Batch_Size = 4
Sequence_Length = 8
torch.manual_seed (1000)

posição = torch.randint (0, len (dados_treino) - Sequence_Length, (Batch_Size,
x = torch.stack ([dados_treino [i:i + Sequence_Length] for i in posição])
y = torch.stack ([dados_treino [i + 1: i + Sequence_Length + 1] for i in posição])

print (x.shape)  #(B, T)
```

```

print (x)
print (y.shape) #(B, T)
print (y)

#mais para a frente
sda = x

torch.Size([4, 8])
tensor([[5383, 161, 237, 87, 357, 83, 4521, 4344],
        [2611, 42, 621, 88, 221, 425, 87, 83],
        [133, 105, 1207, 88, 3633, 153, 88, 981],
        [162, 2043, 83, 1314, 288, 6743, 685, 217]])
torch.Size([4, 8])
tensor([[161, 237, 87, 357, 83, 4521, 4344, 35],
        [42, 621, 88, 221, 425, 87, 83, 879],
        [105, 1207, 88, 3633, 153, 88, 981, 487],
        [2043, 83, 1314, 288, 6743, 685, 217, 196]])

```

3. Embedding

Word Embeddings são representações vetoriais de palavras.

Normalmente, cada token é mapeado para um **vetor de valores reais**, onde a sua posição no espaço vetorial reflete o seu significado. Palavras com significados semelhantes tendem a estar próximas nesse espaço, permitindo ao modelo capturar relações semânticas entre elas.

Neste ponto, introduz-se uma das variáveis fundamentais de uma arquitetura Transformer:

1. **Embedding Dimension** - Define a dimensão dos vetores utilizados para representar cada token e é um dos principais fatores no número total de parâmetros do modelo.

Os Embeddings são parâmetros treináveis, ou seja, são ajustados durante o treino do modelo de forma a melhorar a representação dos tokens no espaço vetorial.

Após a aplicação dos Embeddings passamos a ter um tensor com dimensão (B, T, C).

```

In [ ]: Embedding_Dimension = 16

Embedding = nn.Embedding (tokenizer.get_vocab_size(), Embedding_Dimension) #Pe
#Se usássemos por exemplo len(Tokenizer.tokens) não era correto porque estaria
x_embedding = Embedding (x)
print (x_embedding.shape) #(B,T,C)
print (f"0 Token {x[0,1]} agora é representado pelo vetor:")

```

```
print (x_embedding[0,1]) #Representação vetorial do primeiro token após embedd
torch.Size([4, 8, 16])
0 Token 161 agora é representado pelo vetor:
tensor([ 1.7774,  0.1469, -0.3023, -0.5046, -2.0486, -0.8960,  0.8245,  0.9241,
        -0.3788,  1.7110,  0.8152,  0.2453, -1.2243, -0.1304, -0.2232, -0.284
8],
      grad_fn=<SelectBackward0>)
```

Representação dos Tokens e dos seus respectivos Embeddings

Cada token passa a ser representado por um vetor.

Esta tabela também é excelente para perceber o que **nn.Embedding** faz por trás.

```
In [ ]: import pandas as pd

dados = []

for i in range (8): # for i in range (Sequence Length)
    dados.append ({
        "Token": sda[0,i],
        "Pós Embedding -> Sequence Length [0]": x_embedding[0,i].detach().cpu().
    })

df = pd.DataFrame (dados)
print(df.to_string()) #Melhor maneira de mostrar a tabela completa
```

```
      Token
Pós Embedding -> Sequence Length [0]
0  tensor(5383)  [-0.6550031, -0.8898855, 0.719709, -1.053178, -0.38683298,
-1.0807195, 1.4434127, -0.41286376, 1.5832657, -0.31687176, -0.49571645, -0.765
67215, -0.7344404, -0.6002092, 1.4740021, -2.0729089]
1  tensor(161)   [1.7774044, 0.14689994, -0.3023286, -0.5046191, -2.0486012,
-0.8960302, 0.8245418, 0.9241395, -0.3788303, 1.7110344, 0.81515366, 0.2452607
2, -1.2243475, -0.13037585, -0.22323291, -0.2848408]
2  tensor(237)   [0.44727445, -1.0874941, 0.3502634, 1.3777605, 1.0956962,
-1.7334023, 0.3823119, 0.88006395, 0.066850804, -0.5890923, 0.82223034, -0.2516
813, -1.3240955, -0.9793675, -0.5220944, -0.67033446]
3  tensor(87)    [-0.19269624, -0.14614296, 1.4900721, 0.6922106, -0.1443821
3, -0.09354565, 0.698145, -0.13919245, 1.4728165, 0.2849766, -1.1447818, 0.8410
632, -1.6220355, -0.22049163, -0.8220927, 1.1297927]
4  tensor(357)   [-1.6309177, 1.163017, -0.005374611, -0.7422295, -0.94189405,
-0.06741233, 1.1910526, 0.7179072, -1.8561689, 0.78925943, -1.2319746, -0.01228
97355, 0.13710523, -1.53721, -0.02610584, 1.269864]
5  tensor(83)     [0.23426424, 0.36301392, -0.3973993, -0.9032795, 1.29442,
0.41635293, -0.19065145, 2.1090753, 0.1648092, 0.17884265, 0.7829874, 0.0785082
1, -1.3814574, -1.1322614, -1.0731602, -1.9873365]
6  tensor(4521)   [-1.1509018, -2.4837935, -1.6253139, -0.30007455, -1.16402
2, -0.38122162, 0.03136353, 1.2659864, 0.2795738, -0.866564, -0.4052686, -0.319
98524, -0.8537622, 1.1162541, -0.91794163, 0.456396]
7  tensor(4344)   [-1.2936058, 0.048508823, 0.22302248, 1.323284, 1.6852995,
1.6984859, -0.29643083, 0.2587242, -0.25707203, -1.0180924, -0.21015632, -0.169
2405, -2.0759163, -1.5096134, 1.5811985, 0.51768696]
```

4. Positional Encoding

Positional Encoding é uma técnica que adiciona informação sobre a posição de cada token na sequência.

Isto é necessário porque os Transformers, por si só, não processam os tokens de forma sequencial (como as RNNs) e, por isso, não têm noção da ordem das palavras.

Ao introduzir esta informação, o modelo consegue distinguir entre a posição relativa ou absoluta dos tokens, o que é essencial para compreender a estrutura de uma frase e o significado do texto.

A intuição é a seguinte:

- “Rei ama rainha” $\neq$ “Rainha ama rei” — **a ordem importa!**

No paper original são utilizados encodings fixos.

No entanto, em muitas implementações modernas, são utilizados positional encodings treináveis, que permitem ao modelo aprender a melhor representação para cada posição durante o treino.

Neste projeto foram utilizados encodings treináveis.

```
In [ ]: Positional_Embedding = nn.Embedding (Sequence_Length, Embedding_Dimension) #Pc
        Positional_Encoding = Positional_Embedding (torch.arange (Sequence_Length))
        print (Positional_Encoding.shape)

torch.Size([8, 16])
```

5. Embedding + Positional Encoding

Neste último passo, antes de introduzirmos o mecanismo **Attention**, é necessário combinar os Embeddings dos tokens com os Positional Encodings.

Esta combinação é feita através de uma soma elemento a elemento, permitindo que cada token transporte simultaneamente informação sobre o seu significado e a sua posição na sequência.

Este passo é fundamental, pois prepara a representação final que será utilizada pelo mecanismo **Attention**.

Por essa razão, neste Notebook é apresentado como uma etapa separada

e de especial importância no fluxo da arquitetura.

```
In [ ]: x = x_embedding + Positional_Encoding #Dados que vão entrar no modelo  
print (x.shape)  
print (x[:100])
```

```
torch.Size([4, 8, 16])
tensor([[[[-2.1326e+00, -1.0274e+00,  1.5609e+00, -1.3532e+00, -6.4613e-01,
          -2.1623e+00,  1.2984e+00,  1.2696e+00,  3.0684e+00,  1.0949e+00,
           4.8051e-01, -1.5865e+00, -8.7087e-02, -7.8197e-01,  1.5644e+00,
          -2.4721e+00],
         [ 3.1207e-01,  7.1251e-02, -1.2280e-01, -4.8031e-01, -2.0078e+00,
          -4.0288e-01,  4.6945e-01,  6.5149e-01, -2.8597e-01,  1.5956e+00,
           1.5549e+00,  1.9964e+00, -8.2652e-01,  1.1572e+00, -7.6222e-02,
          -4.6052e-01],
         [-3.2266e-01, -1.5174e+00,  6.4158e-01,  2.0078e+00, -8.5215e-01,
          -1.8710e+00,  4.9936e-01,  7.5644e-01, -4.8657e-01, -1.1196e+00,
           1.8901e+00, -4.3882e-01, -3.1574e+00, -3.6943e-01, -1.9513e+00,
          -6.8812e-02],
         [ 1.7984e+00,  2.5114e-01,  2.0240e+00,  1.3411e+00, -3.1011e-01,
          -1.7915e+00, -3.2437e-01,  1.0267e+00,  7.2454e-01,  1.3891e-01,
          -1.2043e+00,  1.0478e-01,  1.5734e-01,  9.8470e-01,  1.8007e+00,
           1.1580e+00],
         [-2.5748e-02,  5.6563e-01,  6.0615e-01, -1.8509e+00, -1.9459e+00,
           3.9091e-01,  2.6206e+00,  7.5049e-01, -2.4115e+00,  1.2143e+00,
          -2.4037e+00, -8.4490e-01,  1.3295e+00, -2.7965e+00, -5.4023e-02,
           1.3837e+00],
         [ 1.8341e-02, -1.0169e+00,  2.0246e+00,  2.1767e-02,  2.5279e+00,
           5.1874e-02, -9.8382e-01,  2.0296e+00,  7.2189e-01,  2.4914e-02,
          -3.5028e-01,  5.9064e-01, -1.6789e+00, -1.5765e+00, -3.2288e+00,
          -2.5366e+00],
         [-2.4211e+00, -2.0031e+00, -1.4097e+00, -9.3254e-01, -1.5614e+00,
           2.6104e+00, -6.1229e-01, -8.2716e-02,  8.4378e-01,  3.0275e-02,
          -7.6228e-01,  1.4553e-01,  3.3640e-01,  1.3906e+00,  7.3885e-01,
           3.5713e-01],
         [-2.2236e+00,  3.9909e-01,  1.0084e+00,  1.3859e+00,  3.2674e+00,
           8.4834e-01, -1.9474e-02, -1.5733e+00, -1.4568e+00, -3.3868e-01,
          -2.8045e-01,  3.9815e-01, -4.1902e+00, -1.4522e+00,  1.7410e+00,
           1.5805e+00]],

        [[[-2.1547e+00, -1.0077e+00, -1.0762e-02,  1.4027e+00,  1.6435e+00,
          -1.9558e+00, -3.2764e-01,  1.7111e+00,  1.5363e+00,  2.7853e-01,
           1.5562e+00, -1.8906e+00, -1.6287e-02,  1.4911e-01,  6.4962e-02,
           7.4703e-01],
         [-2.4643e+00,  6.2250e-04, -4.1205e-01,  7.9515e-01,  2.6372e+00,
           4.8910e-01,  4.3618e-01,  3.2636e-01, -4.5283e-01,  8.4902e-01,
           3.2990e-01,  3.3208e+00,  9.5547e-01,  1.7854e+00,  1.5089e-01,
          -4.4223e-01],
         [-6.9813e-01,  4.0526e-01, -3.4663e-01, -5.8715e-01, -6.2055e-01,
           3.7257e-01,  1.8432e+00, -7.4408e-01,  2.6791e+00, -2.8279e-01,
           2.4644e+00,  5.1149e-01, -6.0501e-01,  5.6702e-01, -5.9688e-01,
           7.3932e-01],
         [ 2.4924e-03,  1.1306e+00, -1.1554e+00, -1.9690e-01,  7.2453e-01,
          -2.1747e+00, -1.5476e+00,  2.2930e+00,  4.1671e-01, -7.5635e-01,
           1.2852e+00,  6.4491e-02,  2.9848e+00,  1.3372e+00,  3.5073e+00,
           2.0311e+00],
         [ 2.5508e+00, -2.1981e+00, -4.3023e-01,  2.1941e-01, -7.0084e-02,
           2.0977e-01,  1.2170e+00,  7.8650e-01,  9.4511e-01, -4.3300e-01,
           4.1411e-01, -2.7619e+00,  5.9850e-01, -1.0123e+00,  7.3551e-02,
          -9.1344e-01],
```



```

[ 2.2092e-02, -1.2894e+00, 2.4167e+00, 2.6196e+00, 6.9459e-01,
-5.8248e-01, -6.8962e-01, -9.4716e-01, 8.2487e-01, 1.7198e+00,
-7.9213e-01, 2.4948e-01, -5.8971e-01, -8.6665e-01, -3.6128e+00,
-9.8672e-01],
[-1.4628e+00, 3.3458e-01, 1.7057e+00, 5.9744e-02, -5.4177e-01,
2.8981e+00, 5.4491e-02, -1.4879e+00, 2.0370e+00, 1.1818e+00,
-1.5018e+00, 1.3066e+00, -4.3187e-01, 5.3823e-02, 8.3470e-01,
1.0305e+00],
[-6.9571e-01, 7.1360e-01, 3.8797e-01, -8.4064e-01, 2.8765e+00,
-4.3379e-01, 8.6305e-02, 2.7709e-01, -1.0349e+00, 8.5825e-01,
7.1269e-01, 6.4590e-01, -3.4957e+00, -1.0749e+00, -9.1340e-01,
-9.2454e-01]],

[[-5.4508e-01, 1.6177e+00, 2.0805e+00, -7.0952e-01, -2.2653e+00,
7.5993e-01, -8.6992e-01, 2.6796e+00, 1.6353e+00, 2.4374e+00,
8.1573e-01, -1.1075e+00, 1.6412e+00, -1.0054e+00, 5.4329e-01,
-1.4488e+00],
[-3.1412e+00, -2.9742e-01, 1.1546e+00, 7.5206e-02, 3.8805e-01,
7.0535e-01, 2.1910e-01, -4.0766e-01, -1.4741e-01, 1.7152e+00,
1.8143e-01, 1.7364e+00, 4.9674e-01, 1.6227e+00, 1.1965e-01,
-1.0699e+00],
[-1.1000e+00, -2.1783e+00, 1.1486e+00, -7.9715e-01, -1.7330e+00,
-2.3950e+00, -1.4448e+00, 1.4204e-01, -1.9237e+00, -9.0526e-01,
1.3456e+00, 3.2894e-01, -3.2954e+00, 8.3157e-01, -1.5357e+00,
9.1636e-01],
[2.4924e-03, 1.1306e+00, -1.1554e+00, -1.9690e-01, 7.2453e-01,
-2.1747e+00, -1.5476e+00, 2.2930e+00, 4.1671e-01, -7.5635e-01,
1.2852e+00, 6.4491e-02, 2.9848e+00, 1.3372e+00, 3.5073e+00,
2.0311e+00],
[2.6099e+00, -1.0333e+00, -1.2000e+00, 1.1574e+00, -2.2249e+00,
5.1484e-01, -6.0026e-02, -3.7241e-02, -6.7949e-01, 2.4005e-01,
-1.3290e+00, -1.8275e+00, 1.3298e+00, -2.8492e+00, 3.7013e-01,
1.5994e-01],
[-2.7498e+00, 1.4148e-02, 3.2787e+00, 8.2472e-01, 1.0144e+00,
-4.6297e-01, -6.2850e-01, -2.8187e-01, -1.5576e+00, -8.6579e-01,
-3.4405e+00, 2.1858e+00, -1.4920e+00, -1.5421e+00, -2.1749e+00,
2.0243e+00],
[-3.2588e+00, 1.2141e+00, -1.4737e+00, -1.4783e+00, 4.9287e-01,
2.5148e+00, -1.1688e+00, -2.2155e-01, 1.7292e+00, 2.8656e-01,
9.8778e-01, 1.2663e+00, 2.3956e+00, 4.0634e-01, 2.5413e+00,
1.9036e+00],
[-1.3898e+00, 1.0281e+00, 1.8470e+00, -1.5897e+00, 1.6341e+00,
-1.5941e+00, 5.9109e-01, -1.7225e+00, -1.5468e+00, 1.1422e+00,
1.3851e+00, 1.9203e+00, -1.6907e+00, -1.1060e+00, -8.7959e-01,
1.2900e+00]],

[[-3.4274e+00, -1.0819e+00, 1.9336e+00, -2.5626e+00, -1.3190e+00,
-1.2692e+00, -1.4302e+00, 2.0624e+00, 1.0533e+00, -8.9394e-01,
1.2975e+00, 1.9713e-01, -1.2905e+00, 7.8513e-01, -1.3613e+00,
1.0691e+00],
[-2.2057e+00, 1.7729e+00, -9.2113e-01, 5.9539e-01, -1.2942e+00,
2.1965e+00, 3.4189e-01, -6.1835e-01, -6.3409e-01, 6.6684e-01,
1.2613e-01, 3.5773e+00, 2.9784e-01, 5.6845e-01, 1.6432e+00,
-1.0465e+00],

```

```
[ -5.3567e-01, -6.6878e-02, -1.0608e-01, -2.7323e-01, -6.5343e-01,
  2.7877e-01, -7.3603e-02,  1.9854e+00, -3.8861e-01, -3.5169e-01,
  1.8509e+00, -1.0863e-01, -3.2148e+00, -5.2233e-01, -2.5023e+00,
 -1.3858e+00],
[ 8.2235e-01,  5.3447e-01, -1.0564e+00,  4.9327e-01, -7.2042e-01,
 -3.4905e+00, -7.8576e-01, -5.9882e-01, -1.5007e+00, -3.5231e-01,
  3.6037e-01, -1.3240e+00,  2.6158e+00, -3.6630e-01,  2.5997e-01,
  1.0536e+00],
[ 1.6076e+00,  1.2941e-01,  1.0896e+00, -1.9851e-01, -2.0904e+00,
 -3.4635e-01,  5.0276e-01, -5.1271e-01, -4.1062e-01,  6.7576e-02,
 -1.6684e-01, -4.0992e-01,  2.2331e+00, -2.3412e+00,  8.5692e-01,
 -6.9511e-01],
[ 1.1167e+00, -6.4297e-02,  2.4123e+00,  1.0149e+00,  3.7469e-01,
 -1.7795e+00, -1.6081e+00,  1.1543e+00, -5.6712e-01,  3.4555e-02,
 -4.6578e-01, -3.4630e-01, -1.2584e-01,  1.4877e-01, -1.2735e+00,
 -7.9419e-01],
[ 5.7528e-01, -2.6769e-01, -6.6997e-01, -2.3819e+00, -1.1040e+00,
  3.2988e+00,  5.3852e-01, -5.7835e-02, -8.8694e-01,  1.1633e+00,
  1.3193e+00,  1.3366e+00,  3.7956e+00, -1.4289e+00,  7.4370e-01,
 -1.2657e-01],
[ -1.0233e+00,  7.3478e-01, -4.4246e-02, -5.8743e-01,  1.7251e+00,
 -1.4599e+00,  6.7330e-01, -5.4863e-01, -3.9494e-01,  1.6956e+00,
 -1.8484e+00,  1.1650e+00, -2.2970e+00,  7.9011e-01, -6.5044e-02,
  8.2253e-01]]], grad_fn=<SliceBackward0>)
```

Embedding + Positional Encoding

```
In [ ]: dados = []

for i in range (8):
    dados.append ({
        "Token": sda[0,i],
        "Embedding + Positional Encoding -> Sequence Length [0]": x[0,i].detach(
    })

df = pd.DataFrame (dados)
print(df.to_string()) #Melhor maneira de mostrar a tabela completa
```

Token
Embedding + Positional Encoding -> Sequence Length [0]
0 tensor(5383) [-2.1326299, -1.027403, 1.5609097, -1.3531505, -0.6461277, -2.1622794, 1.298405, 1.2695975, 3.06841, 1.0948933, 0.48051322, -1.5864604, -0.08708674, -0.78197026, 1.5643662, -2.4721394]
1 tensor(161) [0.31206906, 0.071251065, -0.122799724, -0.4803076, -2.0078197, -0.40287986, 0.46944943, 0.6514897, -0.28596658, 1.5956165, 1.5549285, 1.9964164, -0.826517, 1.157212, -0.0762216, -0.46051726]
2 tensor(237) [-0.32266152, -1.5173856, 0.641582, 2.007805, -0.85215175, -1.8709887, 0.49936032, 0.75643724, -0.48657063, -1.1196268, 1.8901185, -0.4388229, -3.1574173, -0.36943406, -1.9512589, -0.06881225]
3 tensor(87) [1.7984438, 0.25113615, 2.0239809, 1.3411398, -0.3101055, -1.7914903, -0.3243733, 1.0266868, 0.72454333, 0.13890548, -1.204341, 0.104783416, 0.15733576, 0.9847019, 1.8007424, 1.1580067]
4 tensor(357) [-0.025748134, 0.56562614, 0.60614926, -1.8509035, -1.9459189, 0.39091325, 2.6205678, 0.7504889, -2.4115393, 1.2142706, -2.4037032, -0.8448981, 1.3294585, -2.7964811, -0.05402254, 1.3836869]
5 tensor(83) [0.018341005, -1.0169337, 2.0245614, 0.021766841, 2.5279045, 0.05187431, -0.98381543, 2.0295606, 0.7218938, 0.024914488, -0.35028213, 0.59063596, -1.6788881, -1.5765133, -3.2287664, -2.5366423]
6 tensor(4521) [-2.421051, -2.0030699, -1.4096814, -0.932541, -1.5614052, 2.6103935, -0.6122908, -0.082716465, 0.84378123, 0.030275404, -0.7622752, 0.14553303, 0.3364045, 1.390569, 0.7388522, 0.35712755]
7 tensor(4344) [-2.2235804, 0.39909005, 1.0083953, 1.385923, 3.2673502, 0.8483425, -0.019474417, -1.5732651, -1.4568071, -0.33868057, -0.28044957, 0.39814863, -4.190153, -1.4522073, 1.7409617, 1.5804791]

6. Attention

Attention é um mecanismo que permite ao modelo focar-se em diferentes partes da sequência de entrada ao processar cada token.

De forma análoga à atenção humana, ao ler uma frase não analisamos cada palavra isoladamente, em vez disso, consideramos o contexto relevante para compreender o significado.

Da mesma forma, o mecanismo **Attention** permite ao modelo atribuir diferentes níveis de importância aos vários tokens da sequência, capturando relações e dependências entre eles.

Ao contrário das RNNs, este mecanismo permite acesso direto a todos os Tokens anteriores, facilitando a captura de dependências de longo alcance.

Este é o mecanismo central da arquitetura Transformer e pode ser aplicado de diferentes formas, destacando-se três variantes principais:

- **Self-Attention** - cada token atende a todos os outros tokens da mesma sequência, permitindo capturar relações internas.

- **Masked Self-Attention** - semelhante ao Self-Attention, mas com uma máscara que impede o modelo de “ver o futuro”, sendo essencial para tarefas de geração de texto.
- **Cross-Attention** - permite relacionar duas sequências diferentes, onde uma sequência atende à outra

Neste Notebook será utilizada a variante Masked Self-Attention, característica de modelos do tipo GPT.

Query, Key e Value (Q, K, V)

Attention baseia-se em três representações vetoriais:

- **Query (Q)** - O que ando à procura ?
- **Key (K)** - O que represento ?
- **Value (V)** - Que informação trago comigo ?

A Attention é calculada através da comparação entre **Queries** e **Keys**, permitindo determinar o grau de relevância entre tokens.

Esses pesos são depois aplicados aos **Values** para produzir a saída final.

Vamos ver em código:

```
In [ ]: B,T,C = x.shape #Batch, Sequence Length, Embedding_Dimension
        #print (x.shape)
        Head_Size = C = 16 #Head_Size representa a divisão do vetor #Se tiver um vetor

        #C = Embedding_Dimension #Aplica uma Transformação Linear a partir dos Embeddi
        Query = nn.Linear (C, Head_Size, bias = False)
        Key = nn.Linear (C, Head_Size, bias = False)
        Value = nn.Linear (C, Head_Size, bias = False)

        #Aplico a projeção nos dados
        Q = Query (x) #Y = X * Wq
        K = Key (x) #Y = X * Wk
        V = Value (x) #Y = X * Wv

        print (x.shape)
        print (Q.shape)
        print (K.shape)
        print (V.shape)

torch.Size([4, 8, 16])
torch.Size([4, 8, 16])
torch.Size([4, 8, 16])
torch.Size([4, 8, 16])
```

Heatmap da Representação Vetorial do Embedding e dos respectivos Q, K e V

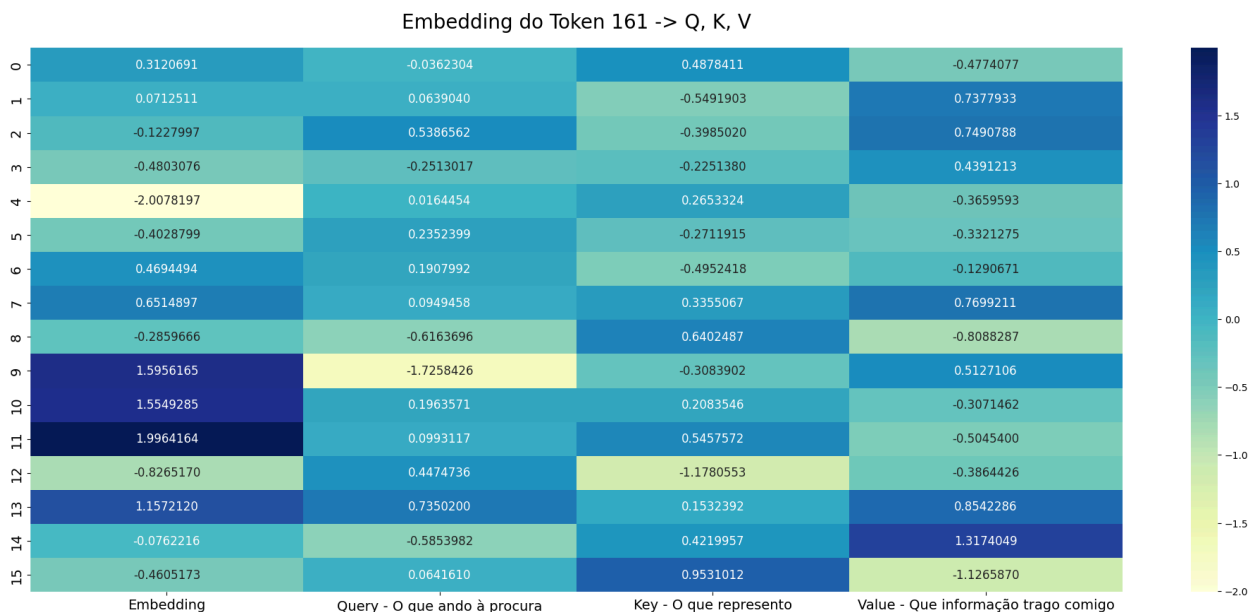
```
In [ ]: import seaborn as sns

df = pd.DataFrame({
    "Embedding": x [0,1].detach().numpy(), #x[0,1] vai selecionar um tensor da
    "Query - O que ando à procura": Q [0,1].detach().numpy(),
    "Key - O que represento": K [0,1].detach().numpy(),
    "Value - Que informação trago comigo": V [0,1].detach().numpy(),
})

plt.figure (figsize = (25, 10))
sns.heatmap(df, cmap = "YlGnBu", annot = True, fmt = ".7f", annot_kws = {"size

plt.xticks (size = 14)
plt.yticks (size = 14)

plt.title(f"Embedding do Token {sda[0,1]} -> Q, K, V", size = 20, pad = 20)
plt.show()
```

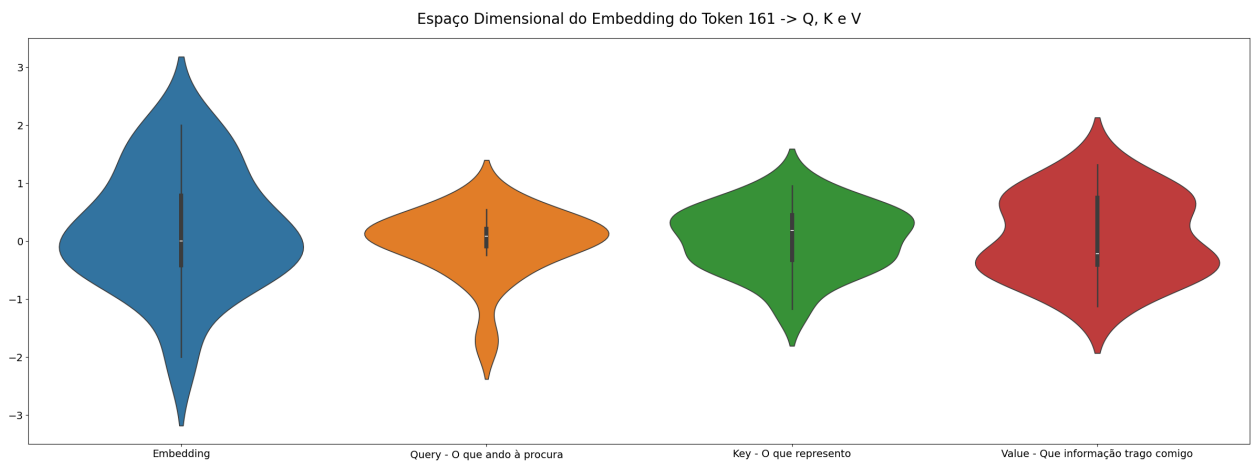


Representação do Espaço Dimensional do Embedding e dos respectivos Q, K e V

```
In [ ]: plt.figure (figsize = (30, 10))
sns.violinplot(df, cmap = "YlGnBu")

plt.xticks (size = 14)
plt.yticks (size = 14)

plt.title(f"Espaço Dimensional do Embedding do Token {sda[0,1]} -> Q, K e V",
plt.show()
```



Scaled Dot-Product Attention (Q @ K)

Scaled Dot-Product Attention é o mecanismo central que permite aos tokens "interagir entre si" dentro de um Transformer.

Em vez de tratar cada posição isoladamente, o modelo passa a construir significado com base no contexto global.

É aqui que deixa de haver apenas transformações individuais (Q, K, V) e passa a haver comunicação entre tokens.

Um ponto importante é que este mecanismo é diferenciável e altamente **paralelizável**, o que o torna eficiente para treino em larga escala.

Além disso, como todos os tokens "conversam" entre si simultaneamente, o modelo consegue capturar dependências de longo alcance — algo que era difícil em arquiteturas anteriores como RNNs.

```
In [ ]: attention_scores = Q @ K.transpose (-2, -1) * (Head_Size ** -0.5) #(B, T, C) @
print (f"Após os Tokens falarem entre si (Q @ K.t) ->\n{attention_scores.shape}")
print (attention_scores[0])

tril = torch.tril (torch.ones(T, T)) #Cria uma máscara
attention_scores = attention_scores.masked_fill (tril == 0, float ('-inf')) #
print (f"\nAplicação de Mask Attention -> \n {attention_scores[0]} \n")

attention_scores = F.softmax (attention_scores, dim = -1) #Softmax para normalizar
print (f"Aplicação de Softmax para normalizar os valores -> \n {attention_scores[0]} \n")

output = attention_scores @ V #Resultado final mostra as relações entre todos os tokens
print (f"Output (attention_scores @ V) -> \n{output.shape} \n{output[0]}")
```

```

Após os Tokens falarem entre si (Q @ K.t) ->
torch.Size([4, 8, 8])
tensor([[ 1.3775,  0.0980,  0.3307,  0.6525,  1.0066,  0.1437,  0.4098,  0.379
1],
        [-0.6385, -0.1751, -0.4454, -0.3017, -0.1070, -0.2943,  0.2465,  0.122
7],
        [-0.1851, -0.7411, -0.7219,  0.0245, -0.5061, -0.4320, -0.7871,  0.253
7],
        [-0.2846,  0.1532, -0.6058, -0.4617, -0.1924, -0.0777,  0.7666, -0.249
1],
        [ 0.2679, -0.3239,  0.9224,  0.0678, -0.4869,  0.7739, -1.5838, -0.105
4],
        [ 0.6897, -0.3412, -0.3966,  0.7617,  0.9681, -0.0941, -0.2419, -0.161
7],
        [ 1.1347,  0.2915,  1.1352,  0.1684,  0.1870,  0.3548,  0.1405,  0.933
4],
        [-0.4693, -0.2360, -0.1936,  1.4472, -1.6055, -1.0163, -0.5373, -0.057
9]]),
      grad_fn=<SelectBackward0>)

```

```

Aplicação de Mask Attention ->
tensor([[ 1.3775,    -inf,    -inf,    -inf,    -inf,    -inf,    -inf,    -in
f],
        [-0.6385, -0.1751,    -inf,    -inf,    -inf,    -inf,    -inf,    -in
f],
        [-0.1851, -0.7411, -0.7219,    -inf,    -inf,    -inf,    -inf,    -in
f],
        [-0.2846,  0.1532, -0.6058, -0.4617,    -inf,    -inf,    -inf,    -in
f],
        [ 0.2679, -0.3239,  0.9224,  0.0678, -0.4869,    -inf,    -inf,    -in
f],
        [ 0.6897, -0.3412, -0.3966,  0.7617,  0.9681, -0.0941,    -inf,    -in
f],
        [ 1.1347,  0.2915,  1.1352,  0.1684,  0.1870,  0.3548,  0.1405,    -in
f],
        [-0.4693, -0.2360, -0.1936,  1.4472, -1.6055, -1.0163, -0.5373, -0.057
9]]),
      grad_fn=<SelectBackward0>)

```

```

Aplicação de Softmax para normalizar os valores ->
tensor([[1.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
        [0.3862, 0.6138, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
        [0.4634, 0.2657, 0.2709, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
        [0.2432, 0.3767, 0.1764, 0.2037, 0.0000, 0.0000, 0.0000, 0.0000],
        [0.2098, 0.1161, 0.4037, 0.1718, 0.0986, 0.0000, 0.0000, 0.0000],
        [0.2200, 0.0785, 0.0742, 0.2364, 0.2905, 0.1004, 0.0000, 0.0000],
        [0.2483, 0.1069, 0.2484, 0.0945, 0.0962, 0.1138, 0.0919, 0.0000],
        [0.0729, 0.0920, 0.0960, 0.4954, 0.0234, 0.0422, 0.0681, 0.1100]]),
      grad_fn=<SelectBackward0>)

```

```

Output (attention_scores @ V) ->
torch.Size([4, 8, 16])
tensor([[ 1.5707,  0.0778,  1.6542,  1.3098, -0.8927,  0.0927,  0.5379, -0.347
2,

```

```

2],      -2.0250, -0.1400, -1.5535, -0.5355,  0.9222, -0.5379,  0.1777, -1.389
5,      [ 0.3136,  0.4829,  1.0986,  0.7754, -0.5694, -0.1681,  0.1285,  0.338
0],      -1.2785,  0.2606, -0.7885, -0.5165,  0.1189,  0.3166,  0.8773, -1.228
7,      [ 0.6224,  0.4813,  1.2126,  0.9511, -0.9693, -0.1021,  0.4991,  0.408
3],      -1.0101,  0.1857, -0.4392, -0.5152,  0.9702,  0.3124,  0.6987, -0.915
9,      [ 0.2292,  0.5845,  0.9164,  0.6195, -0.7478, -0.3118,  0.1794,  0.328
2],      -0.8426,  0.0744, -0.3720, -0.3730,  0.3170,  0.4845,  0.7278, -0.712
9,      [ 0.3222,  0.4489,  0.9282,  0.6566, -1.0510, -0.1276,  0.3031,  0.348
8],      -0.4938,  0.0737,  0.0531, -0.2930,  0.8123,  0.6027,  0.6099, -0.407
8,      [ 0.3841,  0.1402,  0.7869,  0.4426, -0.7156,  0.0837, -0.2202, -0.427
5],      -0.8475, -0.1567, -0.3110,  0.0764, -0.0632,  0.2098,  0.3152, -0.472
6,      [ 0.4644,  0.4395,  0.8923,  0.6507, -0.7866,  0.0162,  0.3453,  0.125
8],      -0.7315,  0.0028, -0.0035, -0.2104,  0.7660,  0.1953,  0.4817, -0.542
3,      [ 0.1677,  0.4036,  0.6002,  0.1236, -0.4790, -0.2467,  0.0518, -0.300
6]],      -0.6162, -0.2943,  0.0055, -0.0492,  0.0200,  0.3587,  0.1350, -0.036
grad_fn=<SelectBackward0>)

```

7. Add & Norm

Layer Normalization (LayerNorm ou Norm) é uma técnica que estabiliza e acelera o treino de Redes Neurais.

Normaliza as ativações dentro de cada exemplo (ao longo das features), garantindo média próxima de 0 e variância próxima de 1.

Ao fazê-lo, reduz a sensibilidade à escala dos dados e à inicialização dos pesos, tornando o treino mais estável e eficiente.

É especialmente eficaz em **RNNs e Transformers**, onde ajuda a mitigar problemas de gradientes instáveis e melhora a convergência.

Residual Connection (ou Add) é uma componente simples mas fundamental em arquiteturas profundas. Consiste em somar a entrada original ao output de uma camada.

Este mecanismo cria um **"um atalho"** para a informação, permitindo que o modelo preserve o sinal original ao longo das camadas.

Como resultado, facilita o fluxo de gradientes, reduz o problema de *vanishing gradients* e permite treinar redes muito mais profundas.

```
In [ ]: LayerNorm = nn.LayerNorm (Embedding_Dimension) #Layer Norm

output = x + LayerNorm (output) #Residual Connection

print ("Output após Add & Norm ->")
print (output.shape)
print (output[0])
```

```
Output após Add & Norm ->
torch.Size([4, 8, 16])
tensor([[ -0.5878, -0.8905,  3.1845, -0.0544, -1.4245, -2.0113,  1.8692,  1.005
7,
         1.2222,  1.0264, -0.9210, -2.0279,  0.8462, -1.2257,  1.7955, -3.718
8],
 [ 0.7522,  0.7570,  1.4560,  0.6297, -2.8484, -0.6614,  0.6412,  1.127
8,
        -2.1551,  1.9590,  0.3966,  1.2326, -0.6687,  1.6018,  1.1816, -2.256
5],
 [ 0.3512, -1.0446,  2.1565,  3.1500, -2.4462, -2.2294,  0.9976,  1.125
9,
        -2.1387, -1.0680,  1.0514, -1.3858, -1.9879, -0.1372, -1.1686, -1.585
9],
 [ 2.0941,  1.2021,  3.5869,  2.3567, -1.8163, -2.4937, -0.1205,  1.506
2,
        -0.9566,  0.1491, -2.0174, -0.7102,  0.6149,  1.7512,  3.0160, -0.282
5],
 [ 0.2582,  1.0927,  2.0529, -0.9252, -4.2965, -0.1880,  2.8680,  1.085
7,
        -3.6931,  1.0215, -2.6359, -1.7411,  2.5537, -1.9743,  0.7821,  0.267
2],
 [ 1.0324, -0.5746,  3.9829,  1.1729,  0.9639,  0.3617, -1.3863,  1.140
3,
        -1.1512, -0.2289, -0.9656,  0.8834, -1.7134, -0.9711, -2.3762, -3.530
6],
 [-1.7414, -1.3742,  0.1439,  0.1276, -3.4364,  2.3748, -0.1757, -0.094
9,
        -0.9187, -0.2327, -1.0382, -0.5528,  1.6320,  1.5208,  1.4538, -1.020
0],
 [-1.6564,  1.7205,  2.9583,  1.8123,  1.7674,  0.0910,  0.1772, -2.501
9,
        -3.3954, -1.2482, -0.2316,  0.2720, -4.0950, -0.2742,  2.2036,  1.494
6]],
      grad_fn=<SelectBackward0>)
```

8. Feed Forward (Multi Layer Perceptron)

Self Attention permite que os tokens recolham informação contextual ao longo de toda a sequência. Mas a Attention, por si só, é limitada pois é uma operação fundamentalmente linear.

Uma **Feedforward Neural Network (FNN ou Multi Layer Perceptron)** fornece exatamente este ingrediente em falta.

Tem como principal função aplicar uma **transformação não linear** à representação de cada token de forma independente. Como se fosse uma camada de «*pensamento*» do Transformer.

Depois de a Attention ter encaminhado a informação entre posições e montado representações sensíveis ao contexto, a MLP processa cada uma dessas representações através de uma pequena Rede Neural capaz de aprender limites de decisão curvos e de alta dimensão.

É aqui que ocorre o verdadeiro trabalho computacional da compreensão da linguagem.

```
In [ ]: MLP = nn.Sequential (
    nn.Linear (Embedding_Dimension, 4 * Embedding_Dimension), #*4 é o que indi
    nn.ReLU (), #Ou GeLU
    nn.Linear (Embedding_Dimension * 4, Embedding_Dimension)
)

output = MLP (output)
print ("Output após Feed Forward ->")
print (output.shape)
print (output[0])
```

```

Output após Feed Forward ->
torch.Size([4, 8, 16])
tensor([[ 0.4505,  0.0339,  0.2105,  0.7285,  0.8972, -0.2566,  0.3162, -0.173
3,
        -0.3165, -0.0533, -0.3739,  0.1908, -0.0578, -0.2758,  0.4873,  0.296
2],
       [ 0.6946,  0.0672, -0.0618,  0.1717,  0.7297, -0.3662, -0.1788, -0.375
7,
        -0.6340,  0.6092, -0.6696,  0.0711,  0.4341, -0.3123,  0.4549,  0.569
5],
       [ 0.3119,  0.0216, -0.3976, -0.0239,  0.0066, -0.4648,  0.0060, -0.449
2,
        -0.0422,  0.1955,  0.0522, -0.2839, -0.0444, -0.1243,  0.2695,  0.365
2],
       [ 1.0024, -0.0025,  0.0219,  0.5037,  0.3772, -0.1794, -0.0381,  0.035
3,
        -0.9326,  0.5536, -0.3062, -0.0164,  0.1522, -0.3994,  0.8721, -0.352
9],
       [ 0.4426,  0.8872, -0.5441,  0.4206,  0.7455,  0.2122,  0.1608, -0.230
9,
        -0.2338,  0.7758, -0.5376, -0.0866,  0.7605, -0.1070,  0.4181,  0.329
2],
       [ 0.0467,  0.5102, -0.3191,  0.4295,  0.2427, -0.5459,  0.3623, -0.321
6,
        -0.6267, -0.3484,  0.1908, -0.3338, -0.3956, -0.5618, -0.1849,  0.384
2],
       [ 0.1580,  0.5016, -0.7133,  0.0048,  0.4181,  0.3526, -0.1349, -0.329
0,
        -0.4650,  0.2505, -0.3851,  0.4629,  0.2173,  0.2402, -0.3650,  0.192
0],
       [ 0.8685,  0.2525, -0.6145,  0.1810,  0.1306, -0.2002,  0.4648, -0.291
9,
        -0.9056,  0.1715,  0.9250, -0.0083, -0.0910, -0.2525,  0.4679,  0.565
0]],
      grad_fn=<SelectBackward0>)

```

9. Add & Norm (Sim, outra vez)

```

In [ ]: LayerNorm = nn.LayerNorm (Embedding_Dimension) #Layer Norm

output = x + LayerNorm (output) #Residual Connection

print ("Output após Add & Norm ->")
print (output.shape)
print (output[0])

```

```

Output após Add & Norm ->
torch.Size([4, 8, 16])
tensor([[[-1.2698, -1.2912,  1.7746,  0.2615,  1.4249, -3.2118,  1.7979,  0.445
2,
          1.8569,  0.5951, -0.8865, -1.4260, -0.5990, -1.8836,  2.5266, -2.026
8],
        [ 1.6731,  0.0536, -0.4238, -0.2684, -0.5696, -1.3728, -0.0888, -0.339
3,
        -1.8445,  2.7690, -0.0818,  1.9874, -0.0380,  0.3056,  0.7582,  0.625
7],
        [ 1.0764, -1.2802, -0.7995,  2.0627, -0.6752, -3.5809,  0.6738, -0.890
9,
        -0.5049, -0.1866,  2.2495, -1.4247, -3.1846, -0.7165, -0.7217,  1.543
5],
        [ 3.7229,  0.0776,  1.9013,  2.2243,  0.3090, -2.3345, -0.5723,  0.931
8,
        -1.3911,  1.1263, -2.0121, -0.0980,  0.3066, -0.0177,  3.4533,  0.252
7],
        [ 0.4843,  2.0646, -1.0784, -1.3897, -0.7622,  0.3885,  2.5039, -0.237
3,
        -3.4059,  2.4654, -4.0739, -1.5120,  2.5465, -3.5089,  0.4016,  1.641
4],
        [ 0.3823,  0.5641,  1.4283,  1.3909,  3.4066, -1.1400,  0.2089,  1.426
6,
        -0.6820, -0.6484,  0.3922, -0.0443, -2.4760, -2.8100, -3.4729, -1.286
5],
        [-2.0565, -0.6939, -3.4400, -0.9891, -0.4817,  3.5101, -1.0528, -1.056
9,
        -0.5043,  0.6492, -1.8906,  1.3483,  0.8640,  1.9810, -0.3341,  0.815
2],
        [-0.6400,  0.7067, -0.4797,  1.5456,  3.3225,  0.2185,  0.7280, -2.393
2,
        -3.5478, -0.1988,  1.4202,  0.1658, -4.5939, -2.1904,  2.4949,  2.535
4]]],
      grad_fn=<SelectBackward0>)
```

10. Language Modelling

Na tarefa de **Language Modelling**, a última camada transforma o estado escondido do modelo numa distribuição de probabilidades sobre o vocabulário.

Para isso, o modelo aplica primeiro uma projeção linear ao vetor de features, convertendo-o num vetor (**logits**) com dimensão igual ao número de palavras possíveis (vocab_size).

De seguida, aplica a função que normaliza os dados, **Softmax**, que transforma esses valores em probabilidades.

O resultado é um vetor onde cada posição corresponde à **probabilidade** de uma

determinada palavra ser o próximo token na sequência, permitindo ao modelo fazer previsões informadas sobre o que vem a seguir.

```
In [ ]: Language_Modelling = nn.Linear (Embedding_Dimension, tokenizer.get_vocab_size(
logits = Language_Modelling (output)

probs = F.softmax (logits, dim = -1)

print ("Logits ->")
print (logits.shape)
print (logits[0])

print ("\nProbabilidades ->")
print (probs.shape)
print (probs[0])
```

```
Logits ->
torch.Size([4, 8, 8000])
tensor([[ -3.6300e-01, -1.0197e+00, -1.3041e-01, ..., -1.0210e+00,
        -2.3394e+00,  4.6157e-02],
        [ 6.5495e-01,  8.0197e-02,  1.0887e+00, ...,  2.3362e-02,
        6.6790e-01,  6.2057e-01],
        [-8.1386e-01,  2.0341e-01, -2.0652e-02, ..., -2.6666e-01,
        -1.8941e-02,  6.0466e-01],
        ...,
        [-1.5682e+00, -5.2607e-01,  3.1441e-01, ..., -1.4652e-01,
        -3.6829e-01, -1.1281e-01],
        [ 3.6124e-01,  2.0598e-01,  2.0447e-04, ..., -2.0601e-01,
        5.2811e-01,  1.0600e+00],
        [-1.3902e+00, -9.8199e-01,  5.4845e-01, ..., -2.3548e+00,
        -8.8252e-01,  1.6234e+00]], grad_fn=<SelectBackward0>)
```

```
Probabilidades ->
torch.Size([4, 8, 8000])
tensor([[5.6156e-05, 2.9122e-05, 7.0861e-05, ..., 2.9084e-05, 7.7811e-06,
        8.4545e-05],
        [1.9056e-04, 1.0725e-04, 2.9404e-04, ..., 1.0133e-04, 1.9304e-04,
        1.8412e-04],
        [3.4945e-05, 9.6644e-05, 7.7244e-05, ..., 6.0399e-05, 7.7377e-05,
        1.4436e-04],
        ...,
        [1.5421e-05, 4.3725e-05, 1.0133e-04, ..., 6.3909e-05, 5.1198e-05,
        6.6100e-05],
        [1.1267e-04, 9.6467e-05, 7.8526e-05, ..., 6.3893e-05, 1.3313e-04,
        2.2661e-04],
        [1.4790e-05, 2.2244e-05, 1.0277e-04, ..., 5.6364e-06, 2.4571e-05,
        3.0110e-04]], grad_fn=<SelectBackward0>)
```

Conclusão

Este Google Colab teve como principal objetivo implementar e compreender a arquitetura Transformer, proposta no paper ***Attention Is All You Need***. Para além da implementação, serviu como uma base prática de aprendizagem, permitindo consolidar conceitos fundamentais.

Ao longo do projeto, foram introduzidos e explorados vários conceitos essenciais, tais como:

- Sequence Length
- Batch Size
- Embedding Dimension
- Head Size
- Number of Heads
- Logits
- Probabilidades

E percorremos estruturas cruciais dos Transformers tais como:

- Tokenization
- Tensor Shape
- Embedding
- Positional Encoding
- Embedding + Positional Encoding
- Attention
- Add & Norm
- Feed Forward
- Add & Norm
- Language Modelling

Estes elementos não são apenas detalhes técnicos, mas sim componentes estruturais que influenciam diretamente o comportamento, a eficiência e a capacidade de generalização do modelo.

No seu conjunto, estes conceitos formam a base do funcionamento dos **LLM's** e estão presentes de forma recorrente no desenvolvimento e aplicação de modelos modernos de **Processamento de Linguagem Natural**.

A compreensão destes elementos é, portanto, fundamental para evoluir de uma perspetiva introdutória para uma utilização mais avançada e

crítica desta arquitetura.